King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 02

Introduction to Complexity Analysis

Term 261

Reading Assignment

- "Data Structures and Algorithms in Java", 2nd Edition, Adam Drozdek, Thomson Learning, ISBN 0-534-49252-5.
  - Chapter 2 (Sections 1 – 8)
  - Not included:
    - Section 2.9: Amortized Complexity
    - Section 2.10: NP-Completeness

Outline

- Computational and Asymptotic Complexity
- Asymptotic Notations
  - Big-O Notation
  - Big-Ω Notation
  - Big-Θ Notations
- Types of Complexities
- Finding Complexity of algorithms:
  - Examples
- The Best, Average, and Worst Cases

How do we Measure Efficiency?

- There are often many different algorithms that can be used to solve the same problem.
  - For example, searching for a key in an array.

- Thus, it makes sense to develop techniques that allow us to:
  - Compare different algorithms with respect to their "efficiency"
  - Choose the most efficient algorithm for the problem

How do we Measure Efficiency?

- The efficiency of an algorithm can be measured mainly in two ways:
  - Time efficiency: How fast?
    - Measures **how much time an algorithm takes to execute**.
    - Goal: Find an algorithm that solves the problem **as quickly as possible**.
  - Space efficiency: How much memory?
    - Measures **how much memory an algorithm needs during execution**.
    - Goal: Find an algorithm that consumes as little memory as possible.
  - An algorithm can be faster but use more memory or use less memory but take more time. Therefore, we often consider both time and space efficiency when comparing algorithms.

Algorithm efficiency = Time efficiency + Space efficiency

How do we Measure Efficiency?

- Experimental Analysis:
  - How?
    - Implement the algorithm
    - Run the algorithm on real inputs.
    - Measure execution time
  - Advantages
    - Easy to find
  - Disadvantages
    - Different measurements on each run
    - Varies widely based on the hardware, operating systems, machines, languages, compilers, and inputs used for testing

```
long startTime = System.currentTimeMillis();          //
record the starting time
/* (run the algorithm) */
long endTime = System.currentTimeMillis();            //
record the ending time
long elapsed = endTime - startTime;                   //
compute the elapsed time
```

Moving Beyond Experimental Analysis

- Goal: develop an approach to analyzing the efficiency of algorithms that is:
  - Independent of the hardware and software environment
- To avoid these dependencies, we analyze algorithms theoretically, without implementing them. Instead of measuring actual time, we:
  - Look at a high-level description of the algorithm
  - Count the number of basic operations it performs
  - Express this count as a function of the input size n
  - This function, usually written as T(n), represents how the algorithm's cost grows as the input size increases (for example, the size of an array or a file).

Construct a mathematical function representing the runtime of an algorithm

Counting the Number of Basic Operations

- We define a set of high-level primitive operations that are independent from the programming language used and can be identified also in the pseudocode.
  - arithmetic operations: e.g., a + b
  - Assignment: e.g., x =10
  - Reading or writing of primitive types
  - Comparison: e.g., x < 12
  - method calls: e.g., calculate(x)
  - return statements from methods. e.g., return sum;
  - array Access: e.g., a[i]

```
temp = x;
x = y;
y = temp;
```

```
x = x + y;
y = x - y;
x = x - y;
```

- We consider an operation such as ++ , += , and *= as consisting of two basic operations.

Counting the Number of Basic Operations: Example 1

- Find the number of basic operations for the following code fragments:
  - Adding two integers

```
x = 10;
y = 5;
z = x + y;
```

→ 1
→ 1
→ 2

T(n) = 4

  - Swapping two integers

```
temp = x;
x = y;
y = temp;
```

→ 1
→ 1
→ 1

T(n) = 3

```
x = x + y;
y = x - y;
x = x - y;
```

T(n) = 6

Counting the Number of Basic Operations: Loops

- If a loop executes **m times**, we count how many times each part of the loop runs.

Part of loop	Number of executions
Initialization	**1** time
Loop condition	**m + 1** times
Loop-index update (i++)	**m** times
Each statement in the loop body	**m** times

```
for (initialization; loop condition; loop-index update) {
      body;
}
```

```
initialization
while (loop condition) {
      body;
}
```

Counting the Number of Basic Operations: Example 3

- Find the number of basic operations for the following method:
  - The number of iterations of the loop is $5 = 5-1+1$

How to find m ?

```
myMethod(){
        sum = 0;
        for(k = 1;  k <=  5; k++){
            sum = sum +  k;
      }
      return sum;
  }
```

→ **1**

→ **1 + 6 + 5*2=17**

→ **5 * 2 =10**

→ **1**

T(n) = 29

We say that $T(n) = c$ where c is a constant

- The initialization statement is executed 1 time
- The loop condition is executed m + 1 times.
- The loop-index update statement is executed m times.
- Each of the statements in the loop body is executed m times.

Counting the Number of Basic Operations: Example 4

- Find the exact number of basic operations in the following program fragment:
  - The number of iterations of the loop is (n-1)-0+1=n

```
double x, y;
x = 2.5 ; y = 3.0;
for(int i = 0; i < n; i++){
    a[i] = x * y;
    x = 2 * x;
    y = y + a[i];
}
```

→	**2**
→	**1 + (n+1)+ (n*2) =3n+2**
→	**3n**
→	**2n**
→	**3n**

$$T(n) = 11n+4$$

- The initialization statement is executed 1 time
- The loop condition is executed m + 1 times.
- The loop-index update statement is executed m times.
- Each of the statements in the loop body is executed m times.

Computational and Asymptotic Complexity

- Since counting the exact number of basic operations
  - is cumbersome, and sometimes impossible,
  - We can always focus our attention on **asymptotic complexity**

Computational Complexity: The general study of the resources (like **time** and **memory**) an algorithm needs. It answers:

- How much time does it take? (time complexity)
- How much memory does it need? (space complexity)

Mathematically representing complexity of an algorithm as a function of the input

Asymptotic Complexity: describes how the complexity (time and memory) grows as the input size **grows very large**

Analysis of algorithm behavior as its input approaches infinity

focuses on the **growth rate**

Computational and Asymptotic Complexity

- It is used when disregarding lower-order terms of a function to express the efficiency of an algorithm. For example, instead of using $T(n) = n^2 + 100n + \log_{10}n + 1{,}000$ we will say: $T(n) \approx n^2$

n	*f(n)*	n^2		100*n*		$\log_{10}n$		1,000	
	Value	Value	%	Value	%	Value	%	Value	%
1	1,101	1	0.1	100	9.1	0	0.0	1,000	90.83
10	2,101	100	4.76	1,000	47.6	1	0.05	1,000	47.60
100	21,002	10,000	47.6	10,000	47.6	2	0.001	1,000	4.76
1,000	1,101,003	1,000,000	90.8	100,000	9.1	3	0.0003	1,000	0.09
10,000	101,001,004	100,000,000	99.0	1,000,000	0.99	4	0.0	1,000	0.001
100,000	10,010,001,005	10,000,000,000	99.9	10,000,000	0.099	5	0.0	1,000	0.00

Figure 2-1 The growth rate of all terms of function $f(n) = n^2 + 100n + \log_{10}n + 1{,}000$

We can describe the function as having an order of growth of n^2

Asymptotic Complexity allows us to ignore constant factors and lower-order terms and focus on the main components of a function that affects its growth

$$f(n) = n^2 + 100n + \log_{10}n + 1{,}000$$

Asymptotic Complexity

- We don't really care about the number of steps/operations.
- What we really care: how the number of steps grows as the size of input grows.
  - e.g. when input size doubles, "the running time quadruples"
  - so, $0.5n^2$ and $500n^2$ are no different! → constant factors do NOT matter!
  - Example: $T_1(n) = 0.5n^2$ and $T_2(n) = 500n^2$

$$\frac{T_1(2n)}{T_1(n)} = \frac{0.5(2n)^2}{0.5n^2} = 4 \qquad \frac{T_2(2n)}{T_2(n)} = \frac{500(2n)^2}{500n^2} = 4$$

- We care about large input sizes
  - For example, We do not study algorithm to search for an element in a 10-elements array.

Asymptotic Complexity

- So, the following functions are of the same class

$$n^2 \qquad 2n^2 + 3n \qquad \frac{n^2}{165} + 1130n + 3$$

- For example, we could call this class $O(n^2)$
- $O(n^2)$ is asymptotic notation
- It is a simplification of the running time
  - It doesn't tell the exact time a program takes in real-world applications.
  - In real-world applications, constant factors matters!

Big-O tells us about the growth rate, not the exact real-world running time.

Big-O Notation: Examples

- Express each of the following using the big O notation.

- $f(n) = 5n + 12$ is $\longrightarrow$ $O(n)$
- $f(n) = 104$ is $\longrightarrow$ $O(1)$
- $f(n) = n^2 + 3n + 112$ is $\longrightarrow$ $O(n^2)$
- $f(n) = n^3 + 1999n + 1337$ is $\longrightarrow$ $O(n^3)$
- $f(n) = n + \sqrt{n}$ is $\longrightarrow$ $O(n)$
- $f(n) = nlog(n) + n$ $\longrightarrow$ $O(nlog(n))$

Big-O Notation

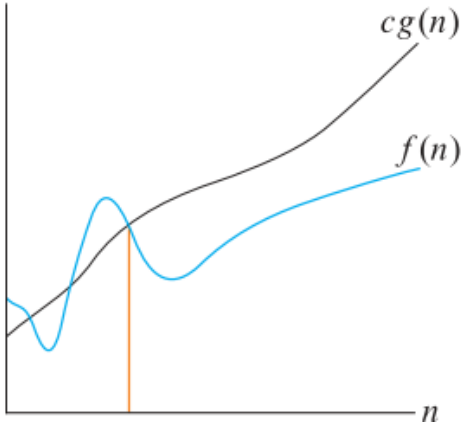
If a running time is $O(f(n))$, this means once n gets large enough, the running time is at most $kf(n)$ for some constant k.

- **Definition 1**: The function f(n) is O(g(n)) if there exists constants $c > 0$ and an integer $N \geq 1$ such that $f(n) \leq c.g(n)$ for all $n \geq N$
  - i.e. g(n) is "upper bound" for f(n).
  - f(n) grows no faster than g(n), asymptotically
- Example:

  $f(n) = 2n^2 + 3n + 1$ $\quad g(n) = n^2$ $\quad f(n)$ is $O(g(n))$

  $2n^2 + 3n + 1 \leq 6n^2$ for all $n >= 1$

  $f(n) \leq c\, g(n) \rightarrow f(n)$ is $O(n^2)$ for all $n \geq 1$ when $c = 6$

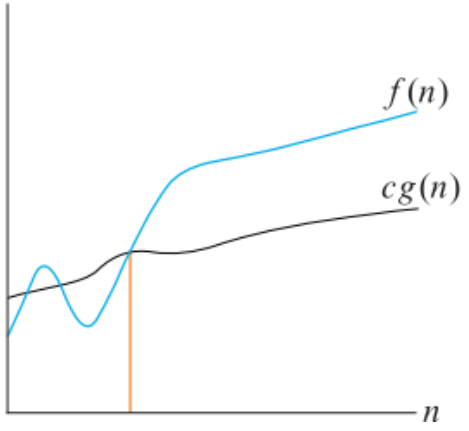


O-notation gives an asymptotic upper bound for a function within a constant factor.

- Usually, we are interested in the tightest upper bound.
- For example, although it is correct to state that $T(n) = 3n^2$ is in $O(n^3)$, we prefer saying that $T(n)$ is in $O(n^2)$.

Big-Ω Notation

- **Definition 2:** The function f(n) is Ω(g(n)) if there exists constants $c > 0$ and an integer $N \geq 1$ such that f(n) ≥ c.g(n) for all n ≥ N
  - i.e. g(n) is "lower bound" for f(n).
  - f(n) grows no less than g(n), asymptotically



Ω-notation gives an asymptotic lower bound for a function within a constant factor.

f (n) is Ω(g(n)) iff g(n) is O(f (n))

- Example:

$f(n) = 2n^2 + 3n + 1 \quad g(n) = n^2 \quad f(n)\ is\ O(g(n))$

$2n^2 + 3n + 1 \geq 2n^2$ for all n >= 1

f(n) ≥ c g(n) ⟶ f(n) is $\Omega(n^2)$ for all n ≥ 1 when c = 2

- Usually, we are interested in the tightest lower bound.
- For example, although it is correct to state that $T(n) = 3n^2$ is in $\Omega(n)$, we prefer saying that $T(n)$ is in $\Omega(n^2)$.

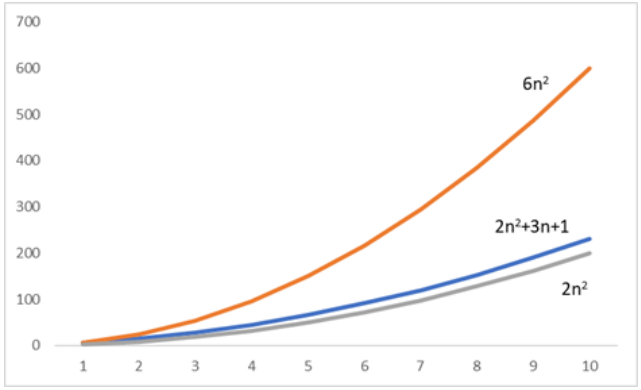
Big-Θ Notation

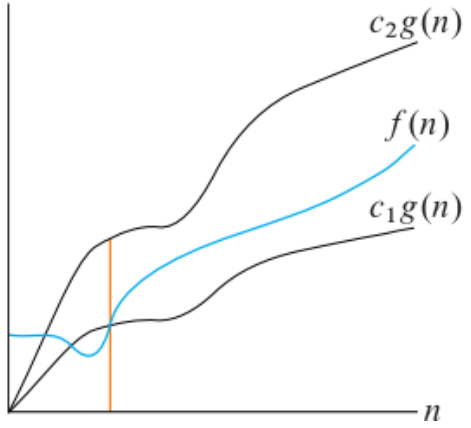
- **Definition 3**: f(n) is in Θ(g(n)) if there exist positive numbers c_1, c_2 and N such that $0 \leq c_1.g(n) \leq f(n) \leq c_2.g(n)$ for all $n \geq N$.
- Equivalently, f(n) is Θ(g(n)) if and only if f(n) is in O(g(n)) and f(n) is in Ω(g(n))
  - i.e. f(n) is approximately equal to g(n), up to a multiplicative constant, for large values of n.
- Example:

$$f(n) = 2n^2 + 3n + 1 \; is \; \Theta(n^2)$$

$$2n^2 \leq 2n^2 + 3n + 1 \leq 6n^2$$

$C_1 = 2$ and $C_2 = 6$





Θ -notation characterizes a tight bound on the asymptotic behavior of a function

When O() and Ω() meet, we indicate this by using Θ() (Big-Theta) notation.

Big-O Notation

- Example: show that $2n^2 + 3n + 1$ is $O(n^2)$

$$f(n) = 2n^2 + 3n + 1 \quad \text{and} \quad g(n) = n^2$$

  - Method 1: Replace lower order terms (positives only) with the higher order terms

$2n^2 + 3n + 1 \leq 2n^2 + 3n^2 + n^2$ for all $n >= 1$

$2n^2 + 3n + 1 \leq 6n^2$

$f(n) \leq c\, g(n) \rightarrow f(n)$ is $O(n^2)$ for all $n \geq 1$ when $c = 6$

- Example: show that $2n^2 - 3n + 1$ is $O(n^2)$

$$f(n) = 2n^2 - 3n + 1 \quad \text{and} \quad g(n) = n^2$$

$2n^2 - 3n + 1 \leq 2n^2 + 1$ for all $n >= 1$

$2n^2 - 3n + 1 \leq 2n^2 + n^2$ for all $n >= 1$

$2n^2 - 3n + 1 \leq 3n^2$

$f(n) \leq c\, g(n) \rightarrow f(n)$ is $O(n^2)$ for all $n \geq 1$ when $c = 3$

Big-Ω Example

- Show that $f(n) = 4n^2 + 5n + 1$ is $\Omega(n^2)$
  - Method 1: Drop the lower order terms (positives only)

$4n^2 + 5n + 1 \geq 4n^2$ for all $n \geq 1$

$f(n) \geq c\, g(n) \rightarrow f(n)$ is $O(n^2)$ for all $n \geq 1$ when $c = 4$

- Show that $f(n) = 4n^2 - 5n + 1$ is $\Omega(n^2)$

$4n^2 - 5n + 1 \geq 4n^2 - 5n$ for all $n \geq 1$

$4n^2 - 5n + 1 \geq 4n^2 - n^2$ for all $n \geq 5$ $\quad -5n \geq -n^2$

$4n^2 - 5n + 1 \geq 3n^2$ for all $n \geq 5$ $\quad -5 \geq -n \rightarrow n \geq 5$

- $f(n) \geq c\, g(n) \rightarrow f(n)$ is $O(n^2)$ for all $n \geq 5$ when $c = 3$

Big-Ω Notation: Examples

- Find c1, c2 and N to show that

$$5n^4 + 3n^3 + 2n\log(n) + 4n + 1 \text{ is } \Theta(n^4)$$

- $5n^4 + 3n^3 + 2n\log(n) + 4n + 1 \leq 5n^4 + 3n^4 + 2n^4 + 4n^4 + n^4$ for all $n \geq 1$
- $5n^4 + 3n^3 + 2n\log(n) + 4n + 1 \leq 15n^4$ for all $n \geq 1$
- So, $f(n) = O(n^4)$ when C1 = 15 and N =1

- $5n^4 + 3n^3 + 2n\log(n) + 4n + 1 \geq 5n^4$ for all $n \geq 1$
- So, $f(n) = \Omega(n^4)$ when C2 = 5 and N =1
- So, $f(n) = \Theta(n^4)$ when C1 = 5 , c2 =15 and N =1

Warnings about O-Notation

- When applying any of these notations (big-O,Ω, and Θ), remember they are approximations that hide some detail that in some cases may be considered important.
  - Big-O notation only gives sensible comparisons of algorithms in different complexity classes when n is large.

- Consider two algorithms for the same task. Which is faster?

Linear: $f(n) = 1000\,n$ When f(n) is faster than g(n) ?

Quadratic: $g(n) = \frac{n^2}{1000}$ When g(n) is faster than f(n) ?

$$g(n) < f(n) \rightarrow \frac{n^2}{1000} < 1000n \rightarrow n^2 < 1000000n \rightarrow n < 1000000$$

Input size n	Faster algorithm
$n < 1,000,000$	Quadratic $g(n) = \frac{n^2}{1000}$
$n = 1,000,000$	Both take the same time
$n > 1,000,000$	Linear $f(n) = 1000n$

Big-Oh vs. Actual Running Time

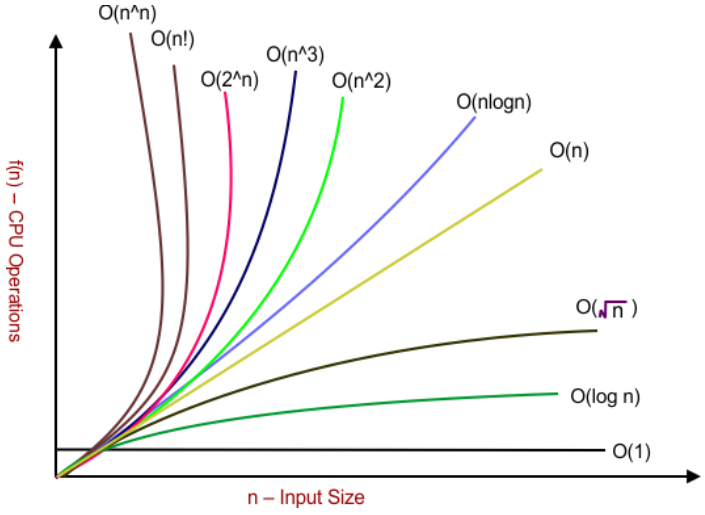
- Let algorithm A and B have running times $T_A(n) = 20n$ and $T_B(n) = 0.2n^2$

  - In the Big-Oh sense, A is better than B
    - This means that, for sufficiently large input size, Algorithm A will generally be faster than Algorithm **B**.

  - But on which data size A outperforms B?
    - $T_A(n) < T_B(n) \rightarrow 20n < 0.2n^2 \rightarrow \quad n>100$
    - Thus, A is better than B for n > 100

Asymptotic-Complexity Classes

Class	Complexity	If you double n, time will	Remarks
Constant	O(1)	unchanged	The execution time remains the same for any number of elements
Logarithmic	O(log(n))	increase slightly	It discards a large quantity of values in each iteration
Linear	O(n)	doubles	Its complexity grows in direct proportion to the size of the input
Quadratic	$O(n^2)$	quadruples	Its complexity is directly proportional to the squared size of the input
Cubic	$O(n^3)$	multiplied by 8	Its complexity is directly proportional to the cubic size of the input



Growth rates

Complexity	$n = 10$	10^2	10^3	10^4	10^5	10^6
$O(1)$	1 μs	1 μs	1 μs	1 μs	1 μs	1 μs
$O(\log n)$	3 μs	7 μs	10 μs	13 μs	17 μs	20 μs
$O(n)$	10 μs	100 μs	1 ms	10 ms	0.1 s	1 s
$O(n \log n)$	33 μs	664 μs	10 ms	133 ms	1.6 s	20 s
$O(n^2)$	100 μs	10 ms	1 s	1.7 min	16.7 min	11.6 days
$O(n^3)$	1 ms	1 s	16.7 min	11.6 days	31.7 yrs	31,709 yrs
$O(2^n)$	1 ms	$\approx 3.17 \times 10^{17}$ yrs	10^{301}	10^{3010}	10^{30103}	10^{301030}

Classes of algorithms and their execution times on a computer executing 1 million operations per second (1 sec = 10^6 µsec = 10^3 msec)

Finding Asymptotic Complexity: Examples

- For iterative-based algorithms, we will focus our attention on counting how many times one or both of the following is executed:
  - The most frequently executed statement; e.g. loop body
  - Element comparisons
  - Example:
    - System.out.println(i) is the most frequently executed statement

```
for( i = 0; i < n; ++i )
   System.out.println(i);
```

Useful Formulas

$$\sum_{i=m}^{n} c = (n - m + 1)\, c$$

$$\sum_{i=1}^{n} (a_i + b_i) = \sum_{i=1}^{n} a_i + \sum_{i=1}^{n} b_i$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} a_i = \sum_{i=1}^{m} a_i + \sum_{i=m+1}^{n} a_i \qquad 1 \le m < n$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\log_b a = \frac{\ln a}{\ln b} \qquad \log_b a = \frac{\log_c a}{\log_c b}, c > 1.$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

$$\log ab = \log a + \log b \qquad \log \frac{a}{b} = \log a - \log b$$

$$\sum_{i=0}^{n} a^i = \frac{a^{n+1} - 1}{a - 1}, a \neq 1$$

$$2^{\log_2 n} = n \qquad a^{\log_a n} = n$$

$$a^{b+c} = a^b . a^c \qquad (a^b)^c = (a^c)^b = a^{bc}$$

Finding Asymptotic Complexity: Examples

- What is the asymptotic complexity of the following program fragments
  - Represent the cost of the for loop in summation form.
  - The main idea is to make sure that we find an iterator that increases/decreases its value by 1.

```
sum = 0
for (i = 0; i < n; i++)
    sum += a[i];
```

$$\sum_{i=0}^{n-1} 1 = (n - 1 - 0 + 1) = \Theta(n)$$

Line	Basic operations
i=0, sum = 0,	2
i < n	n+1
i = i+1	2n
sum += a[i];	3n
Total number of basic operation	T(n)=6n+3 =$\Theta(n)$

iterations is n-1-0+1= n

Finding Asymptotic Complexity: Examples

- What is the complexity of the following program fragments?
  - Represent the cost of the for loop in summation form.
  - The main idea is to make sure that we find an iterator that increases/decreases its value by 1.

```
sum = 0
for (int i = 1; i <= n; i++)
    sum = sum + i;
```

$$\sum_{i=1}^{n} 1 = (n - 1 + 1) = n = \Theta(n)$$

```
sum = 0
for(int i = 1; i<=1000; i++)
    sum = sum + i;
```

$$\sum_{i=1}^{1000} 1 = (1000 - 1 + 1) = 1000 = \Theta(1)$$

```
for (int i = 0; i<n*n; i++)
    sum = sum + i;
```

$$\sum_{i=0}^{n^2-1} 1 = (n^2 - 1 - 0 + 1) = n^2 = \Theta(n^2)$$

Finding Asymptotic Complexity: Examples

- What is the complexity of the following program fragments?

```
for (int i = 1; i <= n; i = i + 2)
        statement1;
```

```
k = (n – 1) / 2
for (int r = 0; r <= k ; r = r + 1)
        i = 1 + 2*r
        statement1;
```

- The important thing is that **i increases by 2, not by 1**.
- I takes the following values: 1 , 3 , 5 , …, 1+2k
- In the for loop above, the last value i takes is n or n-1. n is odd or n is even.
- Here, we can assume without loss of generality that $1 + 2k = n$, $\rightarrow k = \frac{n-1}{2}$
- Therefore, an iterator r goes from 0 to k can be used

$$\sum_{r=0}^{\frac{n-1}{2}} 1 = \frac{n-1}{2} - 0 + 1 = \frac{n+1}{2} = \Theta(n)$$

Finding Asymptotic Complexity: Examples

- Represent the cost of the following for loop in summation form.

```
for (int i = 1; i <= n; i = i + m)
    statement1;
```

If m =1, $\sum_{i=j}^{n} 1 = n - 1 + 1 = n$

- The important thing is that **i increases by m, not by 1**.
- i: 1 , 1 + m , 1 + 2m , …, 1 + km
- In the for loop, the last value i takes is n
- Here, we can assume without loss of generality that 1 + km = n $\rightarrow k = \frac{n-1}{m}$
- Therefore, an iterator r goes from 0 to k can be used

$$\sum_{r=0}^{\frac{n-1}{m}} 1 = \frac{n-1}{m} - 0 + 1 = \frac{n-1}{m} + 1 = \Theta(n)$$

Finding Asymptotic Complexity: Examples

- Represent the cost of the for loop in summation form.
  - For example, consider finding the number of times statements 1 and 2 get executed below:

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        statement1;
```

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        statement2;
```

  - What is the difference between the two loops?

Finding Asymptotic Complexity: Examples

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
            statement1;
```

$$\sum_{i=1}^{n}\sum_{j=1}^{n} 1 = \sum_{i=1}^{n} n = n\sum_{i=1}^{n} 1 = n^2$$

$$\sum_{i=1}^{n}\sum_{j=1}^{n} 1 = \sum_{i=1}^{n}(n-1+1) = \sum_{i=1}^{n} n = n\sum_{i=1}^{n} 1 = n(n-1+1) = n*n = n^2 = \ominus(n^2)$$

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
            statement2;
```

$$\sum_{i=1}^{n}\sum_{j=1}^{i} 1 = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n}\sum_{j=1}^{i} 1 = \sum_{i=1}^{n}(i-1+1) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2+n}{2} = \Theta(n^2)$$

The important thing is that both **i and j increase by 1**

Finding Asymptotic Complexity: Examples

- What is the complexity of the following program fragments?
  - The most frequently executed statement is **S1**

```
for(int i = 1; i < n; i++)
    for(int j = k; j < m; j++)
            S1;
```

$$\sum_{i=1}^{n-1} \sum_{j=k}^{m-1} 1 = \sum_{i=1}^{n-1} (m-1-k+1) = \sum_{i=1}^{n-1} (m-k)$$

$$= (m-k) \sum_{i=1}^{n-1} 1 = (m-k)(n-1-1+1) = (m-k)(n-1)$$

Finding Asymptotic Complexity: Examples

- What is the complexity of the following program fragments?
  - The most frequently executed statement is **sum += a[j];**

```
for (i = 0; i < n; i++) {
    sum = a[0];
     for (j = 1; j <= i; j++)
            sum += a[j];
     System.out.println ("sum is" + sum);
}
```

$$\sum_{i=0}^{n-1}\sum_{j=1}^{i} 1 = \sum_{i=0}^{n-1}(i-1+1) = \sum_{i=0}^{n-1} i = \left(\frac{(n-1)n}{2}\right) = \frac{n^2-n}{2} = \Theta(n^2)$$

Finding Asymptotic Complexity: Examples

- What is the complexity of the following program fragments?
  - The most frequently executed statement is **sum += a[j];**

```
for (i = 4; i < n; i++) {
    sum = 0;
    for (j = i-3; j <= i; j++)
            sum += a[j];
    System.out.println ("sum is"+ sum);
}
```

$$\sum_{i=4}^{n-1}\sum_{j=i-3}^{i} 1 = \sum_{i=4}^{n-1}(i-i+3+1) = \sum_{i=4}^{n-1} 4 = 4\sum_{i=4}^{n-1} 1 = 4(n-1-4+1) = 4(n-4) = \Theta(n)$$

Finding Asymptotic Complexity: Examples

- What is the complexity of the following program fragments?

```
for (i = 0; i <= n ; i+=4)
  for (j = 1; j <= i; j++)
        sum += a[j];
```

- The important thing is that **i increases by 4, not by 1**.
- I takes the following values: 0, 4, 8,…, 4k
- In the for loop, the final value of i is n.
- Assuming n is divisible by 4, $4k = n \rightarrow k = \frac{n}{4}$
  - i = 4r where r is an iterator that goes from 0 to k can be used
- J takes the following values: 1, 2, …, i

$$\sum_{r=0}^{\frac{n}{4}} \sum_{j=1}^{4r} 1 = \sum_{r=0}^{\frac{n}{4}} (4r) = 4\frac{\frac{n}{4}(\frac{n}{4}+1)}{2}$$

Multiplicative Loops

```
for(int i = 1; i <= n; i = i * 2)
        s1;
```

```
for(int i = n; i >= 1; i = i / 2)
        s1;
```

- The important thing is that **i multiplied by 2** (doubled) in each iteration.
- The loop runs for i = 1, 2^1, 2^2, 2^3, …, 2^k
  - The last value I takes is 2^k
  - In the for loop, the final value of i is n
  - For simplicity we assume n is a power of 2. e.g. 16, 32, or 64
  - $n = 2^k \rightarrow k = \log n$
- $i = 2^r$ where r is an iterator that goes from 0 to k, where k = log n
- Therefore, number of times s1 is executed =

$$\sum_{r=0}^{\log n} 1 = \sum_{r=0}^{\log n} (\log n - 0 + 1) = \log n + 1$$

r	i	i
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4
k	n	2^k

Nested Multiplicative Loops

```
for(int i = 1; i <= n; i = i * 2)
    for(int j = 1; j <= n; j ++)
        s1;
```

r	i	j
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4
K	n	2^k

- The loop runs for i = 1, 2^1, 2^2, 2^3, …, 2^k
  - The last value I takes is 2^k
  - In the for loop, the final value of i is n
  - For simplicity we assume n is a power of 2.
  - $n = 2^k \longrightarrow k = \log n$
  - $i = 2^r$ where r is an iterator that goes from 0 to k, where $k = \log n$
- For each value of *i*, the inner for loop runs for j = 1..n
- Intuitively, the number of times s1 is executed is

$$\sum_{r=0}^{\log n} \sum_{j=1}^{n} 1 = \sum_{r=0}^{\log n} (n - 1 + 1) = \sum_{r=0}^{\log n} n \; = n\,(\log n - 0 \; + 1) = n \log n \; + n = \Theta(n \log n)$$

Nested Multiplicative Loops

```
for(int i = 1; i <= n; i = i * 2)
    for(int j = 1; j <= i; j ++)
        s1;
```

Equivalent to

```
for(int r = 0; r <= log n; r++)
    for(int j = 1; j <= 2^r; j++)
        s1;
```

- The loop runs for i = 1, 2^1, 2^2, 2^3, …, 2^k
  - The last value I takes is 2^k
  - In the for loop, the final value of i is n
  - For simplicity we assume n is a power of 2.
  - $n = 2^k \longrightarrow k = \log n$
  - $i = 2^r$ where r is an iterator that goes from 0 to k, where $k = \log n$
- For each value of *i*, the inner for loop runs for j = 1..i
- Intuitively, the number of times s1 is executed is

r	i	j
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4
K	n	2^k

$$\sum_{r=0}^{\log n}\sum_{j=1}^{2^r} 1 = \sum_{r=0}^{\log n}(2^r - 1 + 1) = \frac{2^{(\log n+1)} - 1}{2 - 1} = \frac{2.2^{\log n} - 1}{1} = 2n-1=\Theta(n)$$

$$\sum_{i=0}^{n} a^i = \frac{a^{n+1} - 1}{a - 1}, a \neq 1$$

Finding Asymptotic Complexity: Examples

```
int binarySearch(int[] arr, int key) {
        int lo = 0, mid, hi = arr.length-1;
        while (lo <= hi) {
                mid = (lo + hi)/2;
                if (key < arr[mid])
                        hi = mid - 1;
                else if (arr[mid] < key)
                        lo = mid + 1;
                else return mid; // success
                }
        return -1;      // failure
}
```

Key	#times
In the middle	$\theta(1)$
Not in the array	$\frac{n}{2^{k-1}} = 1 \rightarrow n = 2^{k-1}$ $\rightarrow k - 1 = \log(n)$ $\rightarrow k = \log(n) + 1$

$\theta(\log n)$ in the worst case

- How many times does the loop execute if the key is in the middle?

- How many times does the loop execute if the key is not in the array?

iteration	#elements	#comparisons
1	n	1
2	$\frac{n}{2}$	2
3	$\frac{n}{2^2}$	3
4	$\frac{n}{2^3}$	4
5	$\frac{n}{2^4}$	5
K	$\frac{n}{2^{k-1}}$	k

How to determine complexity of code structures

Consecutive Statements

- $T_1(n)=\Theta(f(n))$ and $T_2(n)=\Theta(g(n))$, then $T_1(n)+T_2(n)=\Theta(\max(f(n),g(n)))$

```
for( i = 0; i < n; ++i )
    a[ i ]--;
for( i = 0; i < n; ++i )
   for( j = 0; j < n; ++j )
       a[ i ] += a[ j ] + i + j
```

$\Theta(n)$

$\Theta(n^2)$

$\Theta(n+n^2)=\Theta(n^2)$

If/Else

- Big-O the running time of the condition plus the larger of the running times of if and else blocks

```
if (x > y ) {
   temp = x;
   x = y;
   y = temp;
   y++
}
else{
 for( i = 0; i < n; ++i )
         ++k;
}
```

$\Theta(1)$

$\Theta(1)$

$\Theta(n)$ in the worst case

$\Theta(1)$ in the best case

$\Theta(n)$

How to determine complexity of code structures

- **Switch:** Big-O(the most expensive case)
  - Big-O or Big-Θ(the most expensive case) in the worst case
  - Big-O or Big-Θ(the cheapest case) in the best case

```
char key;
int[] X = new int[n];
int[][] Y = new int[n][n];
……..
switch(key)  {
   case 'a':
     for(int i = 0; i < X.length; i++)
       sum += X[i];
     break;
   case 'b':
     for(int i = 0; i < Y.length; j++)
       for(int j = 0; j < Y[0].length; j++)
           sum += Y[i][j];
       break;
 }  // End of switch block
```

→ $\Theta(n)$

→ $\Theta(n^2)$

Overall Complexity: $\Theta(n^2)$ **in the worst case**

$\Theta(1)$ **in the best case; neither a nor b**

How to determine complexity of code structures

- If Statement: Take the complexity of the most expensive case :

```
char key;
        int[][] A = new int[n][n];
        int[][] B = new int[n][n];
        int[][] C = new int[n][n];
        ........
        if(key == '+')  {
           for(int i = 0; i < n; i++)
             for(int j = 0; j < n; j++)
                C[i][j] = A[i][j] + B[i][j];
        } // End of if block

        else if(key == 'x')
           C = matrixMult(A, B);

        else
           System.out.println("Error! Enter '+' or 'x'!");
```

Overall complexity

$\Theta(n^3)$ in the worst case

$\Theta(1)$ in the best case

$\Theta(n^2)$

$\Theta(n^3)$

$\Theta(1)$

```
static int[][] matrixMult(int A[][],int B[][])
 int C[][] = new int[A.length][B.length];
 // Multiply the two matrices
 for(i = 0; i < A.length; i++) {
    for(j = 0; j < B[0].length; j++) {
      for(k = 0; k < B.length; k++)
         C[i][j] += A[i][k] * B[k][j];
      }
 }
```

How to determine complexity of code structures

- Sometimes if-else statements must carefully be checked:
  - Θ(if-else) = Θ(Condition)+ Max[Θ(if), Θ(else)]

```
int[] integers = new int[n];
.....
if(hasPrimes(integers) == true)
```

→ $\Theta(n)$

```
    integers[0] = 20;
```

→ $\Theta(1)$

```
else
    integers[0] = -20;
```

→ $\Theta(1)$

```
public boolean hasPrimes(int[] arr) {
        for(int i = 0; i < arr.length; i++)
                .....
                .....
} // End of hasPrimes()
```

Overall complexity = $\Theta(n)$

Is there a need for best case and worst case in the above example?

Best, Average, and Worst-case Complexities

- Best-case complexity analysis:
  - The smallest number of operations carried out for a given input.
- Worst-case complexity analysis:
  - The largest number of operations carried out for a given input.
- Average-case complexity analysis:
  - The number of operations carried out on average for all inputs.

$$\sum_{\text{for each input i}} (\text{Probability of input i} * \text{Cost of input i})$$

- We are usually interested in the worst-case complexity
  - Easier to compute
  - Represents an upper bound on the actual running time for all inputs
  - Crucial to real-time systems (e.g. air-traffic control)

Complexity as a Function of Input Size

- Different inputs of same size have different run times
- E.g., what is run time (number of comparisons) of linear search in an array of n elements?
  - If the item is the first in the list?
  - If it's the last one?
  - If it's not in the list at all?

Case	Description	#Comparisons
Best Case	Item found at the beginning	$\Theta(1)$
Worst Case	Item found at the end or not found	$\Theta(n)$
Average Case	Item may be found at index 0, or 1, or 2, . . . or n – 1	$\frac{(1 + 2 + \ldots + n)}{n} = \frac{n+1}{2} = O(n)$

Finding Asymptotic Complexity: Examples

```
for (i = 0, length = 1; i < n-1; i++) {
  for(i1 = i2 = k = i; k < n-1 && a[k] < a[k+1]; k++, i2++);
  if (length < i2 - i1 + 1)
        length = i2 - i1 + 1;
  System.out.println ("the length of the longest ordered subarray is"+length);
}
```

Case		Description	Complexity
Best	All numbers in the array are in decreasing order	The outer loop is executed n – 1 times, but in each iteration, the inner loop executes just one time. $\sum_{i=0}^{n-2} 1 = n - 2 - 0 + 1 = n - 1$	$\Theta(n)$
Worst	All numbers in the array are in increasing order, `a[k] < a[k+1]` ∀ k	$\sum_{i=0}^{n-2}\sum_{k=i}^{n-2} 1 = \sum_{i=0}^{n-2}(n-2-i+1) = \sum_{i=0}^{n-2}(n-1) - \sum_{i=0}^{n-2} i$	$\Theta(n^2)$
average	In most cases, the arrangement of data is less orderly	Measuring the efficiency in these cases is of great importance. However, it is far from trivial to determine the efficiency in the average cases	

Finding Asymptotic Complexity: Examples

```
length = 1
for (i = 0; i < n-1; i++) {
  i1 = i2 =  i;
  for( k = i; k < n-1 && a[k] < a[k+1]; k++){
  if (length < i2 - i1 + 1)
        length = i2 - i1 + 1;
  i2++
  }
  System.out.println ("the length of the longest ordered subarray is"+length);
}
```

Case		Description	Complexity
Best	All numbers in the array are in decreasing order	The outer loop is executed n – 1 times, but in each iteration, the inner loop executes just one time. $\sum_{i=0}^{n-2} 1 = n - 2 - 0 + 1 = n - 1$	$\Theta(n)$
Worst	All numbers in the array are in increasing order, `a[k] < a[k+1] ∀ k`	$\sum_{i=0}^{n-2}\sum_{k=i}^{n-2} 1 = \sum_{i=0}^{n-2}(n-2-i+1) = \sum_{i=0}^{n-2}(n-1) - \sum_{i=0}^{n-2} i$	$\Theta(n^2)$
average	In most cases, the arrangement of data is less orderly	Measuring the efficiency in these cases is of great importance. However, it is far from trivial to determine the efficiency in the average cases	

Space Complexity

- Algorithms can be classified by their time or space complexities
  - Space complexity: the amount of storage required to run the algorithm beyond the storage used to hold the input.
    - Additional memory proportional to the size of the input
- What is the space complexity of the following algorithms? See next slide

Algorithm	Complexity	Remarks
Linear Search	$\Theta(1)$	It requires a constant amount of additional memory for variables like loop counters and temporary storage, and this space usage does not grow with the size of the input.
Binary Search	$\Theta(1)$	It maintains a constant number of variables, such as low, high, and mid indices.
Insertion Sort	$\Theta(1)$	It is in-place sorting algorithm, meaning. It only requires a constant amount of extra memory for variables like loop counters and temporary storage.
Merge Sort	$\Theta(n)$	It is not an in-place sorting algorithm; During the merge phase, an auxiliary array is typically used to temporarily store the merged elements.

Space Complexity…

```
INSERTION-SORT(A, n)
1  for i = 2 to n
2      key = A[i]
3      // Insert A[i] into the sorted subarray A[1 : i − 1].
4      j = i − 1
5      while j > 0 and A[j] > key
6          A[j + 1] = A[j]
7          j = j − 1
8      A[j + 1] = key
```

```java
public class MergeSort {
  /** The method for sorting the numbers */
  public static void mergeSort(int[] list) {
    if (list.length > 1) {
      // Merge sort the first half
      int[] firstHalf = new int[list.length / 2];
      System.arraycopy(list, 0, firstHalf, 0, list.length / 2);
      mergeSort(firstHalf);

      // Merge sort the second half
      int secondHalfLength = list.length - list.length / 2;
      int[] secondHalf = new int[secondHalfLength];
      System.arraycopy(list, list.length / 2,
        secondHalf, 0, secondHalfLength);
      mergeSort(secondHalf);

      // Merge firstHalf with secondHalf into list
      merge(firstHalf, secondHalf, list);
    }
  }

  /** Merge two sorted lists */
  public static void merge(int[] list1, int[] list2, int[] temp) {
    int current1 = 0; // Current index in list1
    int current2 = 0; // Current index in list2
    int current3 = 0; // Current index in temp

    while (current1 < list1.length && current2 < list2.length) {
      if (list1[current1] < list2[current2])
        temp[current3++] = list1[current1++];
      else
        temp[current3++] = list2[current2++];
    }

    while (current1 < list1.length)
      temp[current3++] = list1[current1++];

    while (current2 < list2.length)
      temp[current3++] = list2[current2++];
  }
```

```java
public class BinarySearch {

  public static int binarySearch(int[] list, int key) {
    int low = 0;
    int high = list.length - 1;

    while (low<=high) {
      int mid = (low + high) / 2;

      if (key == list[mid])
        return mid;

      else if (key < list[mid])
        high = mid - 1;

      else
        low = mid + 1;
    }

    return -low-1; // Now high < low
  }
```

```java
    public static int searchArray(int[] array, int key) {
        for (int i = 0; i < array.length; i++) {
            if (key == array[i])
                return i;
        }
        return -1;
    }
```